

Meeting Assistant Pipeline — Technical Report

Overview

The meeting assistant turns an English meeting recording into a checked meeting record: summary, minutes, agreed decisions, open proposals and action items, each traced back to numbered transcript lines. Language models only propose content; code-side rules and a separate verification pass decide what survives, and anything uncertain is listed as a flag instead of being silently kept or dropped.

The run has six steps, reported in the app as 1/6 to 6/6:

1. **Speech-to-text**: Whisper transcribes the audio and keeps up to 5 hypotheses per segment.
2. **Refinement**: an LLM picks or merges the hypotheses into one corrected line per segment, guarded by code checks.
3. **Stage A, extraction**: an LLM extracts topics, decisions and tasks as JSON, citing line numbers.
4. **Stage B, filter**: deterministic rules drop or downgrade items whose citations do not hold up.
5. **Stage D, verification**: an LLM checks each remaining item against only the lines it cites.
6. **Stage C, writing**: an LLM writes the summary and per-topic minutes from the verified record.

Verification (D) runs before writing (C) on purpose, so the minutes never describe an item that failed the check.

Output file	Contents
<code>raw_transcript.txt</code>	Whisper's top hypothesis per segment, numbered
<code>refined_transcript.txt</code>	The corrected, numbered transcript used by every later stage
<code>meeting_record.md</code>	Summary, minutes, decisions, proposals, action-item table, flags
<code>meeting_record.json</code>	The same items in machine-readable form

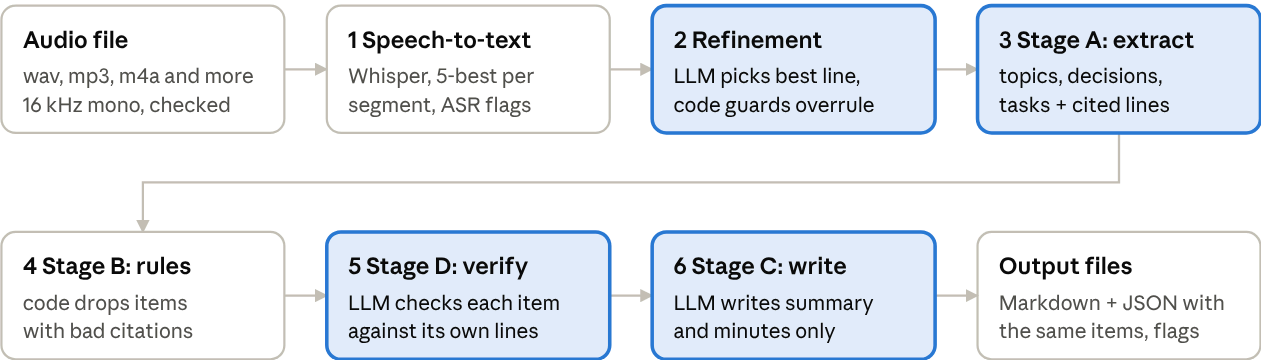
Output file	Contents
<code>nbest_segments.json</code>	Every segment with timestamps, all hypotheses, ASR score and flags

Architecture

One orchestrator, `pipeline.run()`, calls each stage in order and passes a list of segment dicts and a numbered transcript between them. Every model call goes through `llm.py`, so the same stages run either on the local GPU or against hosted APIs without code changes.

Models propose the record; code rules and a verifier decide what stays

☒ language model call (local Qwen or API) ☐ Whisper or plain code



pipeline data flow · 6 steps, 4 of them LLM calls

Only the four tinted steps call a language model; Whisper, the Stage B rules and the output writer are deterministic.

Module	Role
<code>app.py</code>	Gradio web UI on port 7860: upload or record audio, edit the glossary, stream progress, download outputs
<code>pipeline.py</code>	Orchestrator for the six steps; retries with smaller prompts; writes output files; also a CLI

Module	Role
<code>asr.py</code>	Audio loading and checks, Whisper n-best transcription, optional pyannotate speaker labels
<code>refine.py</code>	Refinement prompt and the code guards that can overrule the model
<code>stageA_prompt.py</code>	Extraction prompt, few-shot examples, chunk merging, Stage B rules
<code>document.py</code>	Stage D verification, Stage C writing, Markdown and JSON rendering
<code>llm.py</code>	Model clients: local transformers backend, Groq and Gemini via OpenAI-compatible endpoints, fallbacks, JSON parsing
<code>setup.py</code> , <code>build_stageA_prompts_from_ami.py</code>	One-time build of the AMI few-shot examples in <code>stageA/</code>
<code>eval_utils.py</code> , <code>make_hp_fewshot_prompts.py</code>	Prompt-tuning tools; not used at runtime

Two backends. `LLM_BACKEND=local` loads Hugging Face models on the GPU: a refinement model and a documentation model, of which only one sits in GPU memory at a time. `LLM_BACKEND=api` (or `python app.py --api`) sends refinement to Groq and documentation to Gemini Flash, with Groq models as fallbacks when a quota runs out. Whisper always runs locally.

Pipeline walkthrough

A recording goes through six steps in a fixed order. Each step takes the previous step's output, and every item in the final record points back to numbered transcript lines. Only one language model is loaded on the GPU at a time.

#	Step	Done by	Input	Output
1	Speech-to-text	Whisper (openai/whisper-small)	Audio file (wav, mp3, m4a, ...)	Time-stamped segments, each with up to 5 hypotheses (beam search, 5 beams)
2	Refinement	LLM 1 (Qwen2.5-3B) + code guards	5-best list per segment, glossary, 3 lines of already-corrected context	One corrected line per segment, numbered [id] SPEAKER: text
3	Stage A: extract	LLM 2 (Qwen2.5-7B, 4-bit)	Numbered transcript, in 100-line chunks with 10 lines of overlap	Draft topics, decisions and tasks as JSON, each citing line numbers
4	Stage B: filter	Code only	Draft JSON + transcript	Decisions split into agreed and proposed-only; owners and deadlines kept only if a cited line states them
5	Stage D: verify	LLM 2	Each item + the exact lines it cites	Supported items kept; unsupported items removed and listed as flags
6	Stage C: write	LLM 2	Verified record + transcript grouped by topic	Summary (3-5 sentences) and one minutes paragraph per topic

The output files are written last: raw and refined transcripts, `meeting_record.md`, `meeting_record.json` and `nbest_segments.json`. Decisions and tasks are copied from the verified record into both files, so the Markdown and the JSON always list the same items.

One line traced through (illustrative, from the refinement prompt's worked example):

1. Whisper returns three hypotheses for a segment, best first: "we'll deploy the new build to cube and eighties after the post grass migration", "... to kubernetes after the post grass migration", "will deploy ... to cuban eighties after the postgres migration".
2. Refinement combines them with the glossary entry *Kubernetes* and writes "We'll deploy the new build to Kubernetes after the Postgres migration." The guards accept it: it is close to a hypothesis and adds no number or negation.
3. Stage A reads the line as [14] SPEAKER_01: We'll deploy ... and may draft a task, citing line 14 as evidence.
4. Stage B keeps the task. If the model named an owner that line 14 does not mention, Stage B sets the owner to *unspecified* and adds a flag.
5. Stage D asks whether line 14 really assigns this work. If not, the task is removed and listed under *Flags for review*.
6. Stage C mentions the task in that topic's minutes, and the action-item table cites line 14.

Data

No model was fine-tuned. Data reaches the models only as examples inside the prompts: three AMI excerpts in the Stage A prompt, plus small hand-written sets that cover cases AMI lacks. HyPoradise is not used in any prompt (see Why these prompts).

Data	What it is	Size used	Goes into
AMI Meeting Corpus	Recorded design-team meetings (ES, IS, TS series) with human annotations, converted from XML to JSON with the ami-and-icsi-corpora scripts	3 excerpts of 40 lines, picked from 139 annotated meetings	Stage A few-shot examples
Synthetic Stage A example (SYNTHETIC_EXAMPLE in stageA_prompt.py)	A 12-line hand-written meeting with its gold record	1 example	Stage A few-shot example

Data	What it is	Size used	Goes into
Refinement worked example (<code>DEMO_USER</code> in <code>refine.py</code>)	5 hand-written lines with N-best lists and corrected output	1 example	Refinement prompt
Domain glossary (<code>prompts/glossary.txt</code>)	Technical terms, one per line, editable in the UI	14 terms	Every refinement prompt

AMI annotations mapped to the Stage A record. `build_stageA_prompts_from_ami.py` turns each meeting into numbered lines (consecutive dialogue acts by one speaker are merged, AMI markup is cleaned so it reads like Whisper output) and builds the gold JSON from the annotations:

AMI annotation	Becomes in the Stage A record
Abstractive summary sentences of type "decisions"	<code>decisions[].text</code>
Abstractive summary sentences of type "actions"	<code>tasks[].text</code>
Summary links (which dialogue acts support each summary sentence)	<code>evidence_lines</code> / <code>proposal_lines</code>
Adjacency pairs of type POS (someone agrees with an earlier act)	<code>confirmation_lines</code>
Topic segmentation	<code>topics[]</code> with start and end lines
Role words ("project manager", "industrial designer") or "I'll ..." by that role	Task <code>owner</code>
Phrases such as "by next meeting" or "tomorrow"	Task <code>deadline</code>

Few-shot examples from AMI. The builder slides a 40-line window over each training meeting and scores it: +2 per decision and +3 per task that fit fully inside, -4 per item cut off at the edge, -3 per decision with no visible agreement. This gave 180 candidate excerpts. Three were picked from different scenario series so they cover different cases:

Excerpt	Lines	Topics	Decisions	Tasks	Picked because
ES2008b, lines 400-440	40	3	1	7	Has tasks with a stated owner and a decision
IS1008a, lines 100-140	40	3	2	3	Several decisions with a confirmation line
TS3008a, lines 230-270	40	2	0	4	Tasks with a deadline or a speaker-label owner

The hand-written examples are synthetic, not recorded speech. They exist because AMI never labels a proposal that was *not* agreed or a task with *no* owner, and no public dataset pairs meeting speech with N-best lists in normal casing.

Prompts

The pipeline uses four prompts. All four are instruction prompts with explicit rules, all ask for JSON that follows a strict schema, and all but the writing prompt run at temperature 0 so the same input gives the same output. Two of them carry few-shot examples.

Prompt	Model	Messages sent	Few-shot examples	Output schema	Temperature
Refinement (<code>refine.py</code>)	LLM 1	System rules, then a worked example as a user/assistant pair, then the batch	1 hand-written example of 5 lines	<code>{"lines": [{"id", "text"}]}</code>	0
Stage A extract (<code>stageA_prompt.py</code>)	LLM 2	System rules, then each example as a user/assistant pair, then the transcript	3 AMI excerpts + 1 synthetic meeting	<code>topics, decisions, tasks,</code> each with cited line numbers	0
Stage D verify (<code>document.py</code>)	LLM 2	System rules, then up to 15 items with their cited lines	None	<code>{"results": [{"id", "supported", "reason"}]}</code>	0
Stage C write (<code>document.py</code>)	LLM 2	System rules, then the verified record and the transcript by topic	None	<code>{"summary", "minutes": [{"topic", "text"}]}</code>	0.2

Refinement prompt. It follows the HyPoradise idea: the model sees the N-best list for a line and writes the true transcription. It was changed for meetings in four ways. It corrects 20 lines per call, returning JSON keyed by line id. It adds the glossary. It passes the last 3

corrected lines as read-only context. It keeps normal casing and punctuation. The rules are about what the model must *not* change:

- Fix only words that were likely misheard: sound-alike errors, technical terms, acronyms, product names and people's names.
- Do not rephrase, summarise, translate or correct grammar. Keep fillers, repetitions and false starts.
- Never change numbers, dates, names, negations or commitments (I'll, we will) unless the hypotheses disagree and one is clearly right.
- If unsure, keep hypothesis 1.

The worked example shows one case for each rule: a technical term ("cube and eighties" becomes Kubernetes), a negation kept ("I don't think" even though one hypothesis says "I do think"), a number picked by majority (fifteen, not fifty), disfluencies kept ("we we should"), and names ("pria" becomes Priya, "on air been" becomes Anirban). A user turn looks like this:

Glossary: Kubernetes, PostgreSQL, Redis, CI/CD, OKR

Context (already corrected, do not return these):

[11] So the next item is the deployment.

Lines to correct:

[12]

1. we'll deploy the new build to cube and eighties after the post grass migration

2. we'll deploy the new build to kubernetes after the post grass migration

The code also checks every returned line and can override the model (see Step 2 below for the guard table).

Stage A prompt. The user turn uses the prompting format from the meeting-summarization survey by Rennard et al. ([arXiv 2208.04163](https://arxiv.org/abs/2208.04163), Section 6): Transcript: followed by the numbered lines, then one instruction, "Extract the topics, decisions and tasks from the above transcript. Return only JSON that follows the schema." The few-shot examples use exactly the same format, with the gold JSON as the assistant's reply. The system rules set what counts as each item:

- A decision is "agreed" only if a line shows acceptance or confirmation. Otherwise it is "proposed_only" with no confirmation lines.
- Negation must be kept exactly ("will not use teletext" is not "will use teletext").

- A task's owner is the name, role or speaker label exactly as said, else null. "Someone should look into X" has no owner. Never guess an owner.
- A deadline is recorded only if a time or date is said.
- Every item cites line numbers. Nothing may be invented.

The synthetic example was written to show what AMI cannot: a suggestion that was parked (GraphQL), an agreed decision stated with negation (Android 9 no longer supported), a task with an owner and deadline (Priya, Thursday), and a task with no owner ("someone should update the docs").

Stage D prompt. A checker prompt with no examples. For each item it gets the text, the type, the owner and deadline for tasks, and only the transcript lines that item cites. It answers `supported: true` only if those lines clearly state the item: a decision must be agreed, not just suggested, and a stated owner or deadline must appear on the lines.

Stage C prompt. A writing prompt with no examples. It may use only the verified record and the quoted lines, must not add decisions, tasks, owners, deadlines, numbers or names, and writes "unspecified" where the record does. It runs at temperature 0.2 so the prose reads naturally. Decisions and tasks are not rewritten by this prompt; they are copied from the record.

When a prompt is too large. If the model reports a request too large, Stage A falls back to a smaller prompt: first the synthetic example only, then no examples. Stage C shortens the transcript excerpts from 60,000 to 15,000 to 5,000 characters. Each fallback adds a flag to the record.

Why these prompts

At inference the app sends exactly one prompt per stage: the four described above. Each design choice answers a specific risk in meeting documentation: a wrong correction, an invented owner, or a proposal reported as a decision. No model weights were changed; all behaviour comes from the prompt and the code checks around it.

Refinement prompt: rules + one hand-written meeting example, 20 lines per call.

Choice	Why
The model reads Whisper's 5-best list and writes the true line (the HyPoradise method)	The right word is often in a lower hypothesis ("kubernetes" in hypothesis 2 when hypothesis 1 says "cube and eighties"); the model can pick it using context the recogniser lacks
Rules say what <i>not</i> to change: numbers, negations, names, commitments; keep fillers; if unsure keep hypothesis 1	A flipped "don't" or a changed amount corrupts the decisions and tasks extracted later. A leftover misspelling does far less harm, so the prompt favours leaving text alone
One hand-written meeting example, not HyPoradise examples	HyPoradise references are lowercase with no punctuation, so they would teach the model to drop both. HyPoradise has no meeting speech, and its published template handles one line per call and answers in free text
20 lines per call, JSON keyed by line id	One call per segment would mean hundreds of calls per meeting. Line ids let the code match every answer to its line and fall back to Whisper's line if one is missing
Glossary + the last 3 corrected lines as context	Whisper cannot know team jargon and product names; recent lines keep names and terms consistent across batches
Temperature 0	The same audio gives the same transcript

The code guards (Step 2) back the prompt up: a small 3B model can still drift, so any line that adds a number or negation, or strays far from every hypothesis, reverts to Whisper's line.

Stage A prompt: rules + 3 AMI excerpts + 1 synthetic meeting.

Choice	Why
Transcript: + numbered lines + one instruction (Rennard et al. survey format), with the examples in the same format	A known format for meeting-summarization prompts; identical structure in the examples and the real input shows the model exactly what to produce
3 few-shot excerpts from AMI	AMI is a public meeting corpus whose annotations mark decisions, actions, agreement and topics, so the examples are real meeting talk with gold answers. The text is cleaned to look like Whisper output
Excerpts chosen to differ: tasks with a stated owner, decisions with a confirmation line, a deadline or speaker-label owner, from three scenario series	Each excerpt teaches a different case, rather than three of the same kind
40-line excerpts	Three examples plus a 100-line transcript chunk fit in a small GPU's context
1 hand-written synthetic meeting	AMI never labels a proposal that was <i>not</i> agreed or a task with <i>no</i> owner. Without this example the model sees only agreed decisions and owned tasks, and tends to report every suggestion as a decision and guess owners
Every item must cite line numbers	Stage B and Stage D can then check each item against the exact lines, and readers can find the source in the transcript
Strict JSON schema, temperature 0	Output the code can parse every time, and repeatable results

Stage D and Stage C prompts: rules only, no examples. Both tasks are narrow, so rules are enough. Verification is a separate pass because the extraction model tends to over-claim; a second look at only the cited lines catches items the transcript does not support. The writing prompt is told to use only the verified record, and decisions and tasks are copied from that record, not rewritten, so the minutes cannot add items. It runs at temperature 0.2 so the summary reads naturally.

These prompts were designed from the published methods and the failure cases above; the repository holds no saved measurements comparing them with alternatives.

Development process

The project was built in four stages, each one keeping the prompts and code of the one before.

1. **Design from published methods.** The refinement step follows HyPoradise (LLM correction of ASR N-best lists). The extraction prompt follows the prompting format in Rennard et al.'s meeting-summarization survey. AMI was chosen as the source of gold meeting records because its annotations already mark decisions, actions, agreement and topics.
2. **API version.** The first version called hosted models through OpenAI-compatible endpoints: Groq for refinement (e.g. `llama-3.3-70b-versatile`) and Gemini for documentation. Free-tier limits shaped the design: 20 lines per refinement call and a retry-with-wait wrapper. This backend is still available with `python app.py --api`.
3. **Local GPU version.** Both language models were moved onto the GPU (Qwen2.5-3B in 16-bit, Qwen2.5-7B in 4-bit), loading one at a time so a free Colab T4 (15 GB) is enough. The notebook writes every module as a file and builds the AMI few-shot examples.
4. **Packaged app.** The modules were moved into this folder with a Gradio interface (`app.py`), a one-time `setup.py` that unpacks AMI and rebuilds the few-shot examples, and `run_app.bat` for Windows. On the 6 GB laptop GPU the documentation model was switched to Qwen2.5-3B in 4-bit (see Hardware limits below).

Code-side checks were added wherever the model could not be trusted: meaning guards after refinement, the Stage B rules after extraction, and a separate verify pass before writing. Every check that changes or removes something leaves a flag in the record for a person to review.

Step 1: Speech-to-text (`asr.py`)

Whisper produces time-stamped segments, then each segment is decoded again with beam search to get an N-best list of up to 5 hypotheses, following the HyPoradise approach. The default model is `openai/whisper-small` in fp16 on the GPU.

Input checks (`load_audio`) reject a missing, empty or unsupported file, anything under 1 second, and a silent recording (RMS below $1e-4$), each with a plain-language message. Accepted formats: wav, mp3, m4a, flac, ogg, webm, mp4, aac, wma; non-WAV files need ffmpeg. Audio is resampled to 16 kHz mono with librosa.

Two passes:

1. The Hugging Face ASR pipeline transcribes the whole file in 30-second chunks with `return_timestamps=True` , giving segment boundaries.
2. Each segment's audio is re-decoded with `num_beams=5` , `num_return_sequences=5` , `max_new_tokens=220` . Duplicate hypotheses (ignoring case) are removed. Clips shorter than 0.25 s keep the first-pass text only. On CPU the N-best size drops to 2.

Segment flags set here:

- `low_asr_confidence` : the best beam's sequence score is below -1.0.
- `near_silent_segment` : the clip's RMS is below $5e-4$, where Whisper tends to invent text.

Speaker labels (optional). With `HF_TOKEN` set and the gated `pyannote/speaker-diarization-3.1` model accepted, each segment gets the speaker with the most time overlap, renamed `SPEAKER_00` , `SPEAKER_01` and so on. Without a token the step is skipped and noted.

After transcription the pipeline deletes the Whisper model and empties the CUDA cache so the language models get the whole GPU.

Step 2: Refinement (`refine.py`)

The refinement model sees all hypotheses for a line and writes the most likely true transcription, fixing misheard technical terms, acronyms and names. It must not paraphrase, fix grammar or drop fillers, so the transcript stays a faithful record. Locally this is `Qwen/Qwen2.5-3B-Instruct` ; on the API backend it is a Groq model chosen automatically.

Prompt. A system prompt with the rules, one worked demo (five lines covering a sound-alike term, a negation, a number, repeated fillers and two names), a domain glossary from `prompts/glossary.txt` or the UI, and the 3 previously corrected lines as context. Lines go in batches of 20; the reply is JSON `{"lines": [{"id", "text"}]}` . Batch size and the demo switch come from `prompts/refine_config.json` .

Code guards (`guard()`) can overrule the model, in this order:

Check	Result	Flag
Empty or missing line	Keep hypothesis 1	refine_missing
Line of 3 words or fewer that matches no hypothesis	Keep hypothesis 1	none
Word error rate above 0.5 against every hypothesis	Keep hypothesis 1	refine_drift
Adds a number or negation no hypothesis has, or drops one all hypotheses share	Keep hypothesis 1	meaning_guard
Negation differs from hypothesis 1	Keep hypothesis 1	negation_guard

The result is a numbered transcript, `[12] SPEAKER_01: text`, which every later stage cites by line number.

Steps 3 and 4: Extraction (Stage A) and evidence rules (Stage B)

Stage A asks the documentation model for topics, decisions and tasks as JSON with cited line numbers; Stage B is plain code that removes or downgrades anything the citations do not support.

Stage A prompt (`stageA_prompt.py`) uses the "Transcript: ... + instruction" format from the meeting-summarisation survey by Rennard et al. Its rules:

- **Topics:** title, first line, last line, in order.
- **Decisions:** `agreed` only when a line shows acceptance or an announced decision, with `proposal_lines` and `confirmation_lines`; otherwise `proposed_only` with no confirmation lines. Negation must be kept exactly.
- **Tasks:** owner and deadline only when the transcript states them, else null, each with the lines that state them, plus `evidence_lines`. "Someone should..." has no owner.

Few-shot examples and fallback. The prompt carries three AMI excerpts

(`stageA/stageA_fewshot.json` , built by `setup.py`) plus one hand-written synthetic meeting that covers what AMI lacks: an unaccepted proposal and an ownerless task. If

the request is too large for the model, the pipeline drops the AMI excerpts, then the synthetic example too, and keeps the smaller prompt for the rest of the run; each step down is flagged `smaller_prompt_used`.

Chunking. On the local backend, transcripts longer than 100 lines (`STAGEA_CHUNK_LINES`) are split into chunks with a 10-line overlap. `merge_extractions()` joins the results: decisions and tasks whose texts share at least 60% of their words are merged, their cited lines combined, and `agreed` wins over `proposed_only`; consecutive topics with the same title are joined.

Stage B rules (`stage_b_filter`) the model cannot override:

Rule	Result	Flag
Cited line numbers outside the transcript	Removed from the item	none
Topic with invalid start or end line	Dropped	none
Decision with no valid cited lines	Dropped	<code>dropped_decision</code>
<code>agreed</code> decision with no confirmation line	Moved to proposals	<code>decision_to_proposal</code>
Task with no valid evidence lines	Dropped	<code>dropped_task</code>
Owner or deadline with no cited line	Set to "unspecified"	<code>owner_unsupported</code> , <code>deadline_unsupported</code>
Owner or deadline not found in the cited lines' text	Set to "unspecified"	<code>owner_not_in_text</code> , <code>deadline_not_in_text</code>

The text check matches the whole value, the speaker label, or all of its words longer than 2 letters (ignoring words such as "the" and "next") on a cited line.

Steps 5 and 6: Verification (Stage D) and writing (Stage C)

Stage D removes any item its own cited lines do not clearly state; Stage C then writes prose around the surviving items without being allowed to rewrite them.

Stage D (`document.stage_d_verify`) sends each decision, proposal and task, with its id, type, text, owner, deadline and only the lines it cites, to the documentation model, 15 items per call. The model answers `supported` true or false with a short reason.

- A decision must be shown as agreed or announced, not just suggested; a task must be assigned or volunteered, and its owner and deadline must appear on the cited lines.
- Unsupported items are removed and flagged `unsupported_decision`, `unsupported_proposal` or `unsupported_task`.
- An item the model skips is kept but flagged `not_verified`.
- If a call is too large, it is retried with 4 items per call. If verification fails outright, the run continues with the flag `verification_failed`, because D is a safety net, not a gate.

Stage C (`document.stage_c_write`) gets the verified record as JSON plus the transcript lines of each topic, and returns a 3 to 5 sentence summary and one minutes paragraph per topic. Temperature is 0.2, the only non-zero setting in the pipeline. The transcript budget is 60,000 characters split evenly across topics; on a too-large error it retries at 15,000 and then 5,000 characters, flagged `smaller_prompt_used`. An empty summary is flagged `empty_summary`.

Consistency by construction. Decisions, proposals and tasks are copied from the verified record into both `meeting_record.md` and `meeting_record.json`; the model writes only the summary and minutes. The two files therefore always list the same items, each with its line numbers. All flags from every stage, plus any model switches on the API backend, are collected under "Flags for review".

Model layer (`llm.py`)

Every stage calls `chat_json()` on a client that looks like an OpenAI client, whichever backend is behind it; that one function handles JSON parsing, size errors and rate limits for the whole pipeline.

Local backend (`LocalChatClient`):

- Loads the model with transformers in fp16, or 4-bit NF4 through bitsandbytes when `*_LOCAL_QUANT=4bit`, and uses SDPA attention to cut memory on long prompts.
- Keeps only one model on the GPU: loading a new one unloads the others and empties the CUDA cache, unless `LOCAL_KEEP_ALL=1`.
- Applies the chat template (with thinking disabled for Qwen3-style models) and generates greedily, up to 2,048 new tokens.

- Raises an internal too-large error when the prompt exceeds `LOCAL_MAX_INPUT` (24,000 tokens) or generation hits CUDA out-of-memory. `chat_json` turns that into `RequestTooLarge`, which triggers the smaller-prompt retries in the pipeline.
- With no GPU it refuses to run unless `LOCAL_ALLOW_CPU=1`, and on CPU it first checks that free RAM can hold the float32 weights.

API backend:

- Refinement: Groq, `REFINE_MODEL` if still served, else the best available model from a preference list (Llama 3.3 70B first). A `ModelChain` adds up to 3 other Groq models, each with its own daily budget.
- Documentation: Gemini through its OpenAI-compatible endpoint, defaulting to the newest stable Gemini Flash. With `DOC_FALLBACK_MODEL` set, up to 3 Groq models are chained behind it.
- A daily quota, or a wait longer than 90 s, moves the chain to the next model and records the switch. Shorter per-minute limits are waited out (up to 4 tries).

JSON handling. On APIs, `chat_json` tries a strict `json_schema` response format, then `json_object`, then plain text. Locally there is no constrained decoding, so it asks for JSON only and, on a bad reply, sends one repair turn quoting the invalid output. `parse_json` strips code fences and pulls out the first `{...}` block if the model wrapped it in prose. Auth, connection, retired-model and quota errors become plain-language `LLMError` messages for the UI.

Configuration

Runtime settings live in `.env`; prompt choices live in two JSON files under `prompts/`. Values below are the current ones on this machine.

Setting	Current value	Effect
<code>LLM_BACKEND</code>	<code>local</code>	<code>local</code> GPU models or <code>api</code> (Groq + Gemini)
<code>REFINE_LOCAL_MODEL</code> / <code>_QUANT</code>	<code>Qwen/Qwen2.5-3B-Instruct</code> , <code>4bit</code>	Refinement model
<code>DOC_LOCAL_MODEL</code> / <code>_QUANT</code>	<code>Qwen/Qwen2.5-3B-Instruct</code> , <code>4bit</code>	Model for Stages A, D and C (was 7B; see below)

Setting	Current value	Effect
<code>LOCAL_MAX_INPUT</code>	24000	Largest local prompt, in tokens
<code>LOCAL_ALLOW_CPU</code>	1	Allow running without a GPU
<code>LOCAL_KEEP_ALL</code>	unset	Set to 1 to keep both models loaded on large GPUs
<code>STAGEA_CHUNK_LINES</code>	unset (100 local)	Transcript lines per Stage A call
<code>WHISPER_MODEL</code>	<code>openai/whisper-small</code>	Any Hugging Face Whisper checkpoint
<code>HF_HOME</code>	<code>D:/hf_cache</code>	Model download cache, kept off the nearly full C: drive
<code>MEETING_TZ</code>	<code>Asia/Kolkata</code>	Timezone of the record's timestamp
<code>HF_TOKEN</code>	unset	Enables pyannote speaker labels
<code>GROQ_API_KEY</code> , <code>GEMINI_API_KEY</code> , <code>REFINE_MODEL</code> , <code>DOC_MODEL</code> , <code>DOC_FALLBACK_MODEL</code>	unset	API backend only
<code>prompts/refine_config.json</code>	demo on, batch 20	Refinement prompt variant
<code>prompts/stageA_config.json</code>	<code>ami+synthetic</code>	Stage A starting variant

The app is started with `run_app.bat` (Python 3.11) or `python app.py`, with `--share`, `--port` and `--api` options; `python pipeline.py meeting.mp3 --out <dir>` runs the same pipeline without the UI, and `--segments` skips speech-to-text by feeding prepared hypotheses.

Hardware limits and known issues

On this machine's 6 GB RTX 4050 Laptop GPU, the 7B documentation model does not fit, so Stages A, D and C now run on Qwen2.5-3B in 4-bit.

What went wrong. Extraction failed on a 17-line meeting with "GPU ran out of memory on a 5252-token prompt". Qwen2.5-7B in 4-bit still needs about 5.5 GB, because bitsandbytes leaves the embedding and output layers in 16-bit (about 2.2 GB together). That leaves under 1 GB for the prompt's working memory. The prompt's size comes from the few-shot examples, not the transcript, so lowering `STAGEA_CHUNK_LINES` would not help.

Fix applied on 2026-10-07: `DOC_LOCAL_MODEL=Qwen/Qwen2.5-3B-Instruct`, about 2 GB in 4-bit. Refinement already used the same model and quantisation, so it is loaded once and reused. The trade-off is weaker extraction and verification than 7B would give; Stage B and Stage D limit the damage, since unsupported items are still removed or flagged.

Options for 7B quality:

- The API backend (`LLM_BACKEND=api` with Groq and Gemini keys) uses no GPU memory for the language models.
- A GPU with 8 GB or more, such as Colab's 16 GB T4, runs 7B in 4-bit; the README states 15 GB to run refinement in 16-bit as well.

Other limits:

- The app's startup message and intro table still say "Qwen2.5-7B"; they are hard-coded in `app.py` and do not read `.env`.
- The README lists `REFINE_LOCAL_QUANT` as `16bit` by default; this machine's `.env` sets 4-bit.
- English only: Whisper is forced to `language="en"`.
- Speaker labels need a gated pyannote model and an `HF_TOKEN`; without them every line has no speaker, so owners can only come from names said aloud.
- The local backend has no constrained JSON decoding; malformed output gets one repair attempt, then the stage fails with an error.